

=====

METHOD OVERRIDING IN C++

1. WHAT IS METHOD OVERRIDING?

Method Overriding means:

A child class provides its own version of a method that already exists in the parent class.

In simple words:

Parent class -> has a method Child class -> writes the same method again

The child method overrides the parent method.

2. BASIC EXAMPLE

```
#include <iostream>
using namespace std;

class Parent
{
public:
    void show()
    {
        cout << "Parent Show" << endl;
    }
};

class Child : public Parent
{
public:
    void show()
    {
        cout << "Child Show" << endl;
    }
};

int main()
{
    Child obj;

    obj.show();

    return 0;
}
```

OUTPUT:

```
Child Show
```

Here:

Parent has:

```
show()
```

Child also has:

```
show()
```

The Child's `show()` overrides the Parent's `show()`.

3. WHY DO WE USE METHOD OVERRIDING?

Method overriding is used when:

- Parent class provides a general method.
- Child class needs different behaviour.
- Child wants to implement the method in its own way.

Example:

Parent:

```
void sound()
```

Dog:

```
void sound() -> "Dog barks"
```

Cat:

```
void sound() -> "Cat meows"
```

4. IMPORTANT RULE

For proper method overriding:

Parent and Child should have a method with:

- Same method name
- Same parameters
- Same parameter types

Example:

Parent:

```
void show(int x)
```

Child:

```
void show(int x)
```

This is overriding.

5. SIMPLE EXAMPLE

```
class Animal
{
public:
    void sound()
    {
        cout << "Animal makes sound" << endl;
    }
};

class Dog : public Animal
{
public:
    void sound()
    {
        cout << "Dog barks" << endl;
    }
};

int main()
{
    Dog d;

    d.sound();
}
```

```
    return 0;
}
```

OUTPUT:

Dog barks

6. CALLING PARENT METHOD

If the child overrides the parent method, we can still call the parent method using:

```
ParentClass::method();
```

Example:

```
class Animal
{
public:
    void sound()
    {
        cout << "Animal Sound" << endl;
    }
};

class Dog : public Animal
{
public:
    void sound()
    {
        cout << "Dog Bark" << endl;
    }
};

int main()
{
    Dog d;

    d.sound();           // Child method
    d.Animal::sound();   // Parent method

    return 0;
}
```

OUTPUT:

```
Dog Bark  
Animal Sound
```

7. METHOD OVERRIDING WITH **virtual**

For runtime method overriding, we normally use the keyword:

```
virtual
```

Example:

```
class Animal  
{  
public:  
    virtual void sound()  
    {  
        cout << "Animal Sound" << endl;  
    }  
};  
  
class Dog : public Animal  
{  
public:  
    void sound() override  
    {  
        cout << "Dog Bark" << endl;  
    }  
};
```

8. WHAT IS **override**?

override is a keyword used in the child class.

It tells the compiler:

"This method is supposed to override a parent method."

Example:

```
class Parent  
{  
public:  
    virtual void show()  
    {
```

```
        cout << "Parent" << endl;
    }
};

class Child : public Parent
{
public:
    void show() override
    {
        cout << "Child" << endl;
    }
};
```

Using `override` is recommended because the compiler can catch mistakes.

9. WHY USE `virtual`?

`virtual` is important when using a parent class pointer/reference with a child object.

Example:

```
class Animal
{
public:
    virtual void sound()
    {
        cout << "Animal Sound" << endl;
    }
};

class Dog : public Animal
{
public:
    void sound() override
    {
        cout << "Dog Bark" << endl;
    }
};

int main()
{
    Animal *a;

    Dog d;

    a = &d;

    a->sound();
}
```

```
    return 0;
}
```

OUTPUT:

```
Dog Bark
```

Because `sound()` is virtual, C++ calls the Child's version.

10. WITHOUT `virtual`

```
class Animal
{
public:
    void sound()
    {
        cout << "Animal Sound" << endl;
    }
};

class Dog : public Animal
{
public:
    void sound()
    {
        cout << "Dog Bark" << endl;
    }
};

int main()
{
    Animal *a;

    Dog d;

    a = &d;

    a->sound();

    return 0;
}
```

OUTPUT:

```
Animal Sound
```

Without `virtual`, the parent version is called through the parent pointer.

11. METHOD OVERRIDING vs METHOD OVERLOADING

METHOD OVERRIDING

Parent and Child classes

- Same method name
- Same parameters
- Different implementation

Example:

Parent:

```
void show()
```

Child:

```
void show()
```

METHOD OVERLOADING

Same class usually

- Same method name
- Different parameters

Example:

```
void show()  
void show(int x)  
void show(int x, int y)
```

12. METHOD OVERRIDING vs CONSTRUCTOR OVERLOADING

METHOD OVERRIDING

Parent + Child

Example:

Parent:

```
show()
```

Child:

```
show()
```

CONSTRUCTOR OVERLOADING

Same class

Different constructors with different parameters.

Example:

```
Student()  
Student(int age)  
Student(int age, string name)
```

13. REAL-LIFE EXAMPLE

Parent class:

```
class Animal  
{  
public:  
    virtual void sound()  
    {  
        cout << "Animal makes sound" << endl;  
    }  
};
```

Child class:

```
class Dog : public Animal  
{  
public:  
    void sound() override  
    {  
        cout << "Dog barks" << endl;  
    }  
};
```

```
    }  
};
```

Another child:

```
class Cat : public Animal  
{  
public:  
    void sound() override  
    {  
        cout << "Cat meows" << endl;  
    }  
};
```

Here:

```
Animal -> general sound()  
  
Dog -> Dog's sound()  
  
Cat -> Cat's sound()
```

14. IMPORTANT KEYWORDS

virtual

Used in the parent class to allow runtime polymorphism.

override

Used in the child class to indicate that the method overrides a parent method.

15. QUICK SUMMARY

Method Overriding:

- Happens between Parent and Child classes.
- Child provides its own version of a parent method.
- Method name should be same.
- Parameters should be same.
- **virtual** is used for runtime polymorphism.
- **override** is used in the child class.
- Parent method can still be called using:

```
ParentClass::method();
```

16. EASY WAY TO REMEMBER

OVERLOADING

Same class Same method name Different parameters

↓

```
show()  
show(int)  
show(int, int)
```

OVERRIDING

Parent + Child Same method name Same parameters Different implementation

↓

Parent: show()

Child: show()

END OF METHOD OVERRIDING